Lecture One Assignments:

- Explain the categories of the STRIDE threat-modeling framework, and the security property each one violates with examples.

- In threat prioritization and risk ranking, what do likelihood and impact mean, and how is the risk score calculated?

- What are the three classical security objectives that make up the CIA triad, and what does each one mean?

- Explain the difference between authentication and authorization, with examples.

- What is accountability in the context of application security, and what mechanisms primarily support it?

- What is the difference between input validation using an allow-list versus a block-list, and why should parameterized queries be used instead of string concatenation for SQL?

- Explain the purpose of output encoding, secure password storage, and CSRF protection, including one "bad versus good" example for each.

- Choose correct answer:

  1. Application security is best described as which of the following?
  A) The protection of buildings, data centers, and physical devices
  B) A subset of cybersecurity focused exclusively on securing software throughout its life cycle
  C) The protection of network traffic and connections between systems
  D) The hardening of servers, containers, and cloud accounts

  2. Which of the following is a typical measure for application security?
  A) Firewalls, segmentation, and VPNs
  B) Access badges, locks, and cameras
  C) Threat modeling, secure coding, and SAST/DAST
  D) Host patching and configuration baselines

  3. Which security objective ensures that data is accessible only to authorized parties?
  A) Integrity
  B) Availability
  C) Confidentiality
  D) Accountability

4. A banking application allows an attacker to modify the amount of a transfer after it is submitted. Which security objective is breached?
A) Confidentiality
B) Integrity
C) Availability
D) Authentication
5. Which of the following is a typical failure that leads to a breach of availability?
A) Missing input validation
B) Lack of digital signatures
C) Denial-of-service attacks and resource exhaustion
D) Missing audit trails

6. What does authentication determine?
A) What an authenticated identity is permitted to do
B) Whether the system can provide evidence of who performed an action
C) Who a user, service, or system is
D) Whether data is encrypted in transit

7. In a multi-tenant SaaS billing platform, a support agent can view invoices for every company by changing an account ID in the URL. Which security objective is primarily failing?
A) Authentication
B) Authorization
C) Availability
D) Confidentiality

8. Which mechanisms primarily support accountability?
A) Encryption and access control
B) Rate limiting and redundancy
C) Audit logging, timestamps, and correlation identifiers
D) Passwords and multi-factor authentication

9. Which model moves security activities earlier in the life cycle so feedback arrives while the code is still fresh?
A) Zero-Trust Security
B) Shift-Left Security
C) Defense in Depth
D) Risk-Based Security

10. What is the fundamental purpose of threat modeling?
A) To discover vulnerabilities only after a system has been deployed
B) To shift security analysis from a reactive activity to a proactive activity during system design

C) To replace code review and penetration testing
D) To document functional requirements of an application

11. Which of the following is NOT one of the four major types of element represented in a security-oriented Data Flow Diagram (DFD)?
A) External entities
B) Processes
C) Firewalls
D) Data stores

12. Which STRIDE category involves an attacker pretending to be another user, device, or service?
A) Tampering
B) Repudiation
C) Spoofing
D) Elevation of Privilege

13. Which STRIDE category corresponds to a violation of availability?
A) Information Disclosure
B) Denial of Service
C) Tampering
D) Repudiation

14. Using the scoring formula Risk Score = Likelihood × Impact, what score does a threat receive if likelihood is High (3) and impact is Medium (2)?
A) 5
B) 6
C) 9
D) 3

15. Which secure coding practice keeps SQL code separate from user data so that input can never change the meaning of the query?
A) Output encoding
B) Parameterized queries
C) Secure defaults
D) CSRF protection

- Explain the following concepts with exapmples:
  1- XSS
  2- SQL Injection
  3- API Endpoint
  4- CSRF

- Choose the correct answer:

1. What secure coding practice does the “good” example demonstrate?

BAD EXAMPLE
```
age = request.form["age"]
```
GOOD EXAMPLE
```
age = int(request.form["age"])
if not (0 < age < 120):
raise ValueError("Invalid age")
```

A) Output encoding
B) Input validation with type and range checking
C) Parameterized queries
D) Secure password storage

2. What is the primary security benefit of the “good” example?

BAD EXAMPLE
```
query = f"SELECT * FROM users WHERE name = '{name}'"
cursor.execute(query)
```
GOOD EXAMPLE
```
cursor.execute("SELECT * FROM users WHERE name = %s", (name,))
```

A) It encrypts the database connection
B) It keeps SQL code separate from user data so input cannot change the query’s meaning
C) It validates the length of the name parameter
D) It logs all database queries for auditing

3. What attack does the “good” example prevent?

BAD EXAMPLE
```
return f"<div>{user_comment}</div>"
```
GOOD EXAMPLE
```
import html
safe = html.escape(user_comment)
return f"<div>{safe}</div>"
```

A) SQL injection
B) Cross-site scripting (XSS)
C) Cross-site request forgery (CSRF)
D) Denial of service

4. Why is the “good” example superior to the “bad” example?

BAD EXAMPLE

```
import hashlib
hashed = hashlib.md5(password.encode()).hexdigest()
```

GOOD EXAMPLE

```
import bcrypt
hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
```

A) MD5 is faster and therefore more secure

B) bcrypt is a slow, salted hashing algorithm designed for passwords, while MD5 is fast and unsuitable

C) bcrypt stores passwords in plain text

D) MD5 uses a salt while bcrypt does not

5. What security improvements does the “good” example demonstrate?

BAD EXAMPLE

```
session_id = "user123_session"
response.set_cookie("session", session_id)
```

GOOD EXAMPLE

```
import secrets
session_id = secrets.token_urlsafe(32)
response.set_cookie("session", session_id, httponly=True, secure=True,
samesite="Strict")
```

A) Using a predictable session ID for easier debugging

B) Using a random session ID and setting HttpOnly, Secure, and SameSite cookie attributes

C) Storing the session ID in the URL instead of a cookie

D) Disabling session expiration to improve usability

6. What security flaw does the “good” example fix?

BAD EXAMPLE

```
invoice = get_invoice(invoice_id)
return invoice
```

GOOD EXAMPLE

```
invoice = get_invoice(invoice_id)
if invoice.owner_id != current_user.id:
abort(403)
return invoice
```

A) It encrypts the invoice data before returning it

B) It verifies ownership so a user cannot access another user’s invoice

C) It validates the invoice ID format

D) It logs the access attempt for auditing

7. What is the primary risk of the “bad” example?

```
BAD EXAMPLE
API_KEY = "sk-12345-secret"
GOOD EXAMPLE
import os
API_KEY = os.environ["API_KEY"]
```

A) The API key is too short to be secure

B) Hard-coded secrets can be committed to Git and exposed to anyone with repository access

C) Environment variables are less secure than hard-coded values

D) The API key cannot be rotated

8. What is the security rationale for the “good” example?

```
BAD EXAMPLE
return str(exception)
GOOD EXAMPLE
logger.error("DB error", exc_info=True)
return "Something went wrong. Please try again."
```

A) It hides the error from the user and logs details internally, preventing information disclosure

B) It shows the full error to help users troubleshoot

C) It logs sensitive data to the user-facing response

D) It disables error handling entirely

9. What does the “good” example enforce?

```
BAD EXAMPLE
app.run(host="0.0.0.0", port=80)  # HTTP only
GOOD EXAMPLE
app.run(host="0.0.0.0", port=443, ssl_context=("cert.pem", "key.pem"))  # HTTPS
```

A) It disables all network access

B) It encrypts data in transit using HTTPS/TLS

C) It encrypts data at rest in the database

D) It uses a custom encryption algorithm

10. What principle does the "good" example follow?

BAD EXAMPLE

```
def check_access(user):
       if user.is_admin:
       return True
       return True  # default allow
```

GOOD EXAMPLE

```
def check_access(user):
       if user.is_admin:
       return True
    return False  # default deny
```

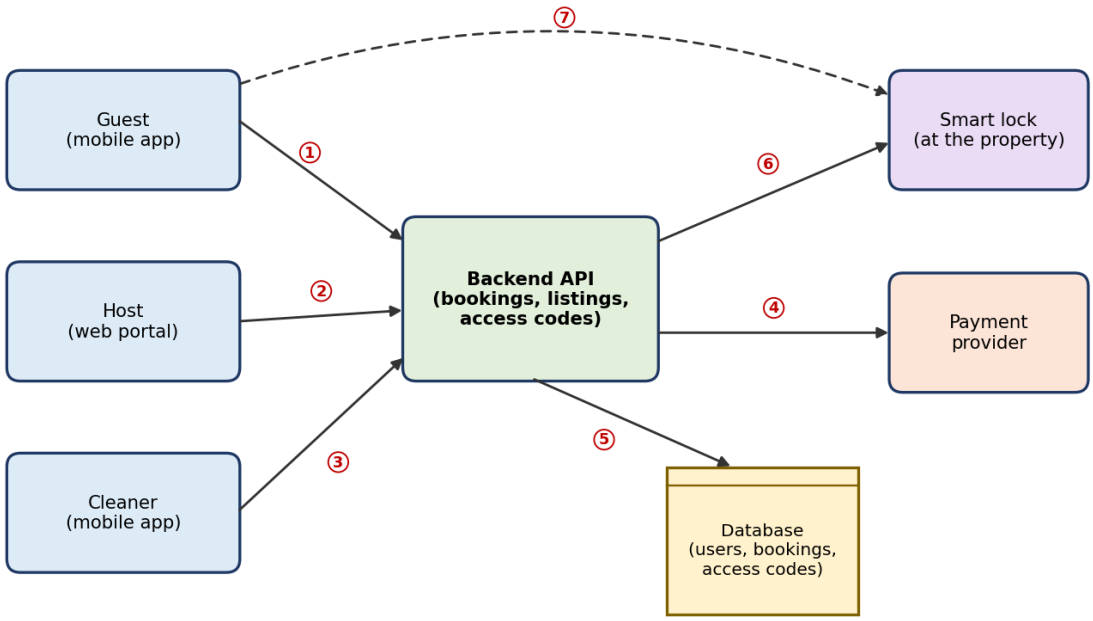
A) Fail open (allow access when in doubt)

B) Fail closed (deny access by default)

C) Least privilege for administrators only

D) Defense in depth

- Suppose you are the cyber security engineer for StayLock, an app for renting homes with smart door locks as shown in the diagram below. The system works like this:
  - Guests use a mobile app to book a stay, pay by card, and unlock the door with their phone over Bluetooth.
  - Hosts use a web portal to manage listings, prices, bookings, and door codes.
  - Cleaners use a mobile app to request a temporary door code for each cleaning visit.
  - A backend API connects everyone and stores users, bookings, and access codes in a database.
  - A third-party payment provider handles card charges.
  - A smart lock at each property receives access codes and reports lock event.



Now, answer the following:

1. **On the diagram, draw three trust boundaries.**
2. **Apply STRIDE. Write one threat for each STRIDE letter (6 in total). Each threat must name the affected component or flow number.**